

OMI-Port from First Principles

Deriving a Universal Declarative Port Interface Without Runtime Authority

Status: Canonical Derivation Draft

Layer: First-Principles / Port Authority / Adapter Boundary

Purpose: Explain why OMI-Port exists and how it is derived

Authority Boundary: Derivation and declaration only. Validation and receipt accept.

0. Starting Question

The starting question is:

How can one thing declare a source-target relation
without pretending it has connected the source to the target?

This is the root problem of OMI-Port.

A normal port often implies activity:

open port
connect port
bind port
listen on port
send through port
receive from port

But OMI needs a port before activity.

It needs a way to say:

this source may be related to that target

without saying:

this source is connected to that target

Therefore, OMI-Port begins from a negative axiom:

A port declaration is not a connection.

Everything else follows from that.

1. First Principle: Declaration Must Precede Connection

Before anything can connect, it must be named.

Before it can be named safely, its scope must be bounded.

Before its scope can be bounded, the system must separate readable description from authority.

So the first derivation is:

connection requires declaration
declaration requires scope
scope requires boundary
boundary requires non-authority

Therefore:

OMI-Port must be declarative before it is connective.

This means OMI-Port cannot begin as a socket, filesystem mount, LoRa stream, DOM handle, canvas object, or OS call.

It must begin as a dormant source-target candidate.

Canonical statement:

OMI-Port declares a possible relation.
It does not activate that relation.

2. Second Principle: A Universal Port Cannot Assume a Universal Runtime

A universal runtime would have to know how to actually operate every environment:

```
filesystems
networks
serial ports
LoRa radios
DOM nodes
canvas graphs
documents
binary streams
sensor events
hardware adapters
```

That would make OMI-Port either impossible or unsafe.

Different platforms have different rules:

```
Linux filesystem permissions
browser sandbox permissions
embedded-device memory limits
radio duty-cycle limits
network firewalls
serial-device handshakes
canvas type rules
validation laws
receipt laws
```

So the standardized layer cannot be the runtime behavior.

The standardized layer must be the interface shape.

Derivation:

```
universal runtime behavior is impossible
universal declaration shape is possible
therefore standardize the declaration shape
and leave runtime behavior downstream
```

Canonical statement:

OMI-Port standardizes the port interface.
It does not standardize the operating authority behind the port.

3. Third Principle: A Port Needs Source and Target

A port is not just a name.

A port has direction.

At minimum, it needs:

source
target

But a raw source-target pair is not enough.

The source and target must be scoped so the route is not ambiguous.

So we derive:

$\text{Port} = \text{Source} \times \text{Target}$

But in OMI, raw source and target must become scoped source and scoped target:

$\text{Port} = \text{SourceScope} \times \text{TargetScope}$

A scope must say:

what namespace is being named
how wide the scope is
where the interpretation stops

This gives the URI-CIDR idea:

`<uri>#/<prefix-length>`

Examples:

```
file:///repo/docs/spec.md#/64
https://example.com/api/v1#/40
dom://document/main/button#/32
lora://915/node/alpha#/16
serial://ttyUSB0#/8
canvas://board/card/7#/24
omi://scope/fs/gs/rs/us#/32
```

The URI names.

The CIDR suffix scopes.

The OMI route transforms.

Validation accepts or rejects.

Canonical rule:

```
URI names.
CIDR scopes.
OMI routes.
Validation accepts.
```

4. Fourth Principle: Source-Target Needs a Transform

If OMI-Port only stored:

```
source
target
```

then it would be a table.

But OMI-Port is not just a table.

It must derive a route.

The route is the interpolation between source and target.

Therefore, OMI-Port needs a transform:

```
Transform(source, target) → route
```

But there is not only one possible transform. Source-target relations may mean:

```
intersection  
union  
difference  
equivalence  
source implies target  
target implies source  
canonical closure
```

So the transform needs a selector.

That selector is the F* Gauge Family.

Derivation:

```
source-target pair needs interpolation  
interpolation needs operator dialect  
operator dialect needs gauge  
therefore OMI-Port uses F* as transform selector
```

Canonical formula:

```
PortTensor_G(source, target) → route candidate
```

or:

```
T_G(source, target) = OMI-IMO place-value route
```

Where:

```
G      = F* gauge/operator dialect  
source = declared source scope  
target = declared target scope  
route  = non-authoritative OMI-IMO candidate
```

5. Fifth Principle: The Output Must Be Canonical Topology, Not Historical Argument Slots

The old LL/MM/NN form was useful historically because it expressed a three-slot transition model:

```
LL = lens / local / selector  
NN = antecedent / source  
MM = consequent / target
```

That is useful as a mnemonic for:

```
lens(source → target)
```

But it begins from argument shape.

OMI-Port cannot canonically begin from arguments.

It must begin from memory topology.

The canonical replacement is:

```
LL/MM/NN → FS/GS/RS/US
```

This replacement is not a rename. It changes the authority model from “arguments first” to “scope topology first,” where OMI begins as a scoped memory envelope rather than a function call.

Therefore, the public OMI-Port output must use:

```
FS/GS/RS/US
```

not:

```
LL/MM/NN
```

Derivation:

```
port needs scoped topology  
LL/MM/NN gives transition slots
```

FS/GS/RS/US gives memory scope
therefore OMI-Port emits FS/GS/RS/US

Canonical statement:

LL/MM/NN may explain the old transform intuition.
FS/GS/RS/US is the canonical port topology.

6. Sixth Principle: The Route Needs Folded and Unfolded Forms

Humans need a compact mnemonic.

Machines need an expanded separator surface.

So the route needs two visible forms.

Folded form:

omi---imo

Unfolded form:

o---o/---/?---?@---@

These are not two different systems.

They are two renderings of the same OMI-IMO route.

The compact frame `omi---imo` and expanded relation form `o---o/---/?---?@---@` are identified as canonical frame forms in the pre-language control envelope.

Derivation:

route needs mnemonic
mnemonic folds the relation
route also needs machine-readable fields
expanded surface unfolds the relation
therefore OMI-Port supports both folded and unfolded forms

Canonical interpretation:


```
omi---imo
  folded route name

o---o/---/?---?@---@
  unfolded separator surface
```

The machine-expanded route is:

```
o-FS-GS-RS-US/FS/GS/RS/US?REGISTER?STACK@CAR@CDR
```

This is the form OMI-Port derives.

7. Seventh Principle: A Port Route Needs a Frame

A route cannot be only text.

It needs a bounded place-value frame.

The OMI-IMO frame is derived as:

```
ruler half
+
payload/pair half
```

The ruler half is:

```
S0 S1 S2 S3 S4 S5 S6 S7
```

Each **S** field is 16 bits:

```
8 × 16 bits = 128 bits
```

The payload/pair half is:

```
REGISTER STACK CAR CDR
```

Each field is 32 bits:

$$4 \times 32 \text{ bits} = 128 \text{ bits}$$

Together:

$$128 \text{ bits} + 128 \text{ bits} = 256 \text{ bits}$$

The existing pre-language authority document states this canonical 256-bit division: the first half is the FS/GS/RS/US ruler cascade, and the second half is the REGISTER/STACK/CAR/CDR payload-pair cascade.

Derivation:

```
route needs bounded fields
scope side needs 8 × 16-bit ruler fields
relation side needs 4 × 32-bit fields
therefore route frame is 256 bits
```

Canonical frame:

```
S0=FS
S1=GS
S2=RS
S3=US
S4=FS '
S5=GS '
S6=RS '
S7=US '
REGISTER
STACK
CAR
CDR
```

The primed fields are the inverse/carrier-side IMO scope.

8. Eighth Principle: A Port Needs Pre-Language Staging

Before readable text, OMI needs a control envelope.

The minimal control alphabet is:

```
0x00  NUL
0x1C  FS
0x1D  GS
0x1E  RS
0x1F  US
0x20  SP
```

These define:

```
origin
file scope
group scope
record scope
unit scope
readable boundary
```

The pre-language control document defines this alphabet and states that `0x1E` may witness closure but is not acceptance; validation and receipt accept.

So the port cannot begin as readable text alone.

It begins as staged scope.

Derivation:

```
readable route needs prior boundary
boundary needs control alphabet
control alphabet stages memory
therefore OMI-Port inherits the pre-language envelope
```

Canonical pre-header:

```
G 00 1C 1D 1E 1F 20 G
```

Default canonical version:

```
FF 00 1C 1D 1E 1F 20 FF
```

Meaning:

```
G    gauge family
00   origin
1C   file scope
1D   group scope
1E   record scope / closure witness
1F   unit scope
20   readable boundary
G    gauge closure
```

The gauge pre-header recognizes and stages a carrier, but does not accept state; this is consistent with the OMI citation invariant that recognition, citation, closure, and projection are not acceptance.

9. Ninth Principle: Dynamic Port Creation Needs Gauge, Not Runtime

If a port route can be derived in multiple ways, the derivation must be declared.

But declaration must still not connect.

The F* Gauge Family solves this.

It gives sixteen possible operator rows:

```
F0 . . FF
```

while keeping the pre-language spine fixed:

```
G 00 1C 1D 1E 1F 20 G
```

So the gauge is dynamic.

The scope topology is canonical.

Derivation:

```
source-target interpolation needs dynamic operator
dynamic operator needs stable enclosure
F* gives dynamic low-nibble dialect
FS/GS/RS/US gives stable scope spine
```

Thus:

```
F* = transform dialect
FS/GS/RS/US = memory topology
omi---imo = folded route
o---o/---/?---?@---@ = unfolded route
```

Canonical statement:

```
OMI-Port uses F* to choose the transform.
OMI-Port uses FS/GS/RS/US to encode the route.
```

10. Tenth Principle: Chirality Is a Bit, Not an Authority

The ASCII pair:

```
'O' = 0x4F
'o' = 0x6F
```

differs by:

```
0x20
```

So a resolver can compute:

```
chirality_bit = byte & 0x20;
flipped       = byte ^ 0x20;
```

This gives a portable sign/lens/chirality operation.

The insight generalizes to ASCII case pairs:

```
A ↔ a
B ↔ b
X ↔ x
O ↔ o
```

But this is not authority.

It is transform information.

Derivation:

```
route needs direction
direction may be represented by chirality
ASCII case gives portable 0x20 chirality
but chirality does not validate
therefore chirality is a candidate transform bit
```

Canonical statement:

```
O/o carries chirality.
Chirality guides route interpretation.
Validation and receipt accept.
```

11. Eleventh Principle: A Port Must Preserve the Authority Boundary

At every layer, OMI-Port must preserve the same invariant:

```
accepted = 0
validated = 0
receipted = 0
```

until downstream validation and receipt authority acts.

The adapter boundary document states that OMI-Lisp candidates convert into typed candidate trees with preserved atom spans, preserved source spans, and non-authoritative flags, and that only downstream validation/receipt authority may set `accepted`, `validated`, or `receipted` to `1`.

Therefore OMI-Port must obey:

```
parsed does not mean valid
scoped does not mean accepted
gauged does not mean validated
connected-looking does not mean connected
projected does not mean receipted
```

Derivation:

```
port declaration may look like connection
therefore flags must remain false
otherwise declaration becomes false authority
```

Canonical invariant:

```
Declaration is not authority.
Connection requires authority.
Validation and receipt accept.
```

12. Twelfth Principle: The Universal Adapter Must Be Dormant

The user asked whether OMI-Port should become:

```
a standardized declarative port
a URI-CIDR notation
a filesystem/networking system
a BusyBox-like universal adapter
an OSI adapter that does not actually connect
```

From first principles, the answer is:

```
standardize the dormant adapter interface
do not standardize runtime behavior as authority
```

A dormant adapter may expose:

```
filesystem-like names
network-like names
serial-like names
LoRa-like names
DOM-like names
canvas-like names
sensor-like names
```

But it must not perform:

```
open
connect
```

```
send
receive
mount
bind
listen
project
validate
receipt
```

Derivation:

```
universal runtime is unsafe
universal declaration is safe
adapter should face many domains
but connect to none by itself
```

Canonical statement:

```
OMI-Port may face OSI.
OMI-Port does not become OSI.
```

13. Final Derived Object Model

From these principles, OMI-Port derives these objects:

```
PortScope
  a URI-CIDR or OMI scope declaration

PortTensor
  a gauge-selected source-target interpolation rule

PortRoute
  a derived OMI-IMO place-value route candidate

PortBinding
  a declared candidate binding

PortPipe
  an authorized active stream, outside OMI-Port v0
```

Only the first four belong to OMI-Port v0.

`PortPipe` is explicitly outside v0 because it implies active runtime authority.

Derivation:

```
scope names
tensor transforms
route frames
binding declares
pipe connects later
```

Canonical rule:

```
A binding may be declared.
A pipe must be authorized.
```

14. Final Derived Pipeline

The full OMI-Port derivation is:

```
source surface
↓
source URI-CIDR scope
↓
target surface
↓
target URI-CIDR scope
↓
F* gauge selection
↓
PortTensor_G(source, target)
↓
FS/GS/RS/US route topology
↓
omi---imo folded route
↓
o---o/---/?---?@---@ unfolded route
↓
OMI candidate
↓
adapter candidate
↓
future validation authority
```

↓
future receipt authority

Nothing in the pipeline connects.

Nothing in the pipeline accepts.

Nothing in the pipeline receipts.

It only derives an inspectable route candidate.

15. Derived C Shape

The minimal C shape follows directly from the derivation:

```
typedef struct {
    const char* uri;
    unsigned prefix_len;
} OMI_PortScope;

typedef struct {
    unsigned char gauge; /* F0..FF */

    OMI_PortScope source;
    OMI_PortScope target;

    unsigned short fs;
    unsigned short gs;
    unsigned short rs;
    unsigned short us;

    unsigned short fs_i;
    unsigned short gs_i;
    unsigned short rs_i;
    unsigned short us_i;

    unsigned int reg;
    unsigned int stack;
    unsigned int car;
    unsigned int cdr;

    int accepted;
    int validated;
```

```
    int receipted;  
} OMI_PortRoute;
```

Transform function:

```
int omi_port_transform(  
    const OMI_PortScope* source,  
    const OMI_PortScope* target,  
    unsigned char gauge,  
    OMI_PortRoute* out  
);
```

Required output:

```
accepted  = 0  
validated = 0  
receipted = 0
```

Required non-behavior:

```
do not connect  
do not transmit  
do not mount  
do not validate  
do not receipt
```

16. Why This Is Not Project-Dependent

The actual backends are project-dependent:

```
real filesystem backend  
real network backend  
real serial backend  
real LoRa backend  
real DOM backend  
real canvas backend  
real validation backend  
real receipt backend
```

But the declarative shape is not project-dependent.

The universal standardized layer is:

```
scope grammar
gauge selection
route derivation
candidate flags
adapter boundary
```

Therefore:

```
OMI-Port should be standardized.
OMI-Port runtime backends should remain project-dependent.
```

This keeps the canon portable without giving false authority to any runtime.

17. First-Principles Summary

Starting from the need to declare a source-target relation without connecting it, we derive:

1. A port declaration is not a connection.
2. A universal runtime is unsafe, but a universal declaration shape is safe.
3. A port requires source and target scopes.
4. Source-target scopes require a transform.
5. Dynamic transforms require a gauge.
6. Canonical route output requires FS/GS/RS/US, not LL/MM/NN.
7. Human and machine surfaces require folded and unfolded OMI-IMO forms.
8. A bounded route requires the 256-bit OMI-IMO frame.
9. Pre-language staging requires G 00 FS GS RS US SP G.
10. Chirality may guide transform but cannot validate.
11. Authority flags must remain false.
12. Runtime connection belongs downstream.

Therefore the derived canon is:

```
OMI-Port is a dynamic F*-gauged source-target transform
that derives canonical FS/GS/RS/US OMI-IMO place-value route candidates
without connecting, validating, accepting, projecting, or receipting them.
```

18. One-Line First-Principles Canon

Because a source-target relation must be nameable before it is connectable, OMI-Port standardizes a dormant F*-gauged URI-CIDR transform into FS/GS/RS/US OMI-IMO route candidates while preserving accepted=0, validated=0, receipted=0 until downstream authority acts.